

트랜잭션

트랜잭션 (Transaction)

MySQL의 동시성에 영향을 미치는 잠금, 트랜잭션, 트랜잭션의 격리수준에 대해서 공부한 내용을 정리했다.

이번 장에서는 **트랜잭션**에 대해서 알아보려고 한다.

트랜잭션이란?

트랜잭션은 작업의 완전성을 보장해주는 것이다.

즉, 논리적인 작업 셋을 모두 완벽하게 처리하거나 처리하지 못할 경우에는 원 상태로 복구해서 작업의 일부만 적용되는 현상이 발생하지 않게 만들어주는 기능이다.

잠금 vs 트랜잭션 vs 격리수준

이 세 가지 개념은 서로 비슷해 보이지만 명확히 다른 역할을 한다.

잠금 (Lock)

- 동시성을 제어하기 위한 기능
- 동시에 같은 데이터를 망가트리지 않도록 해준다
- 만약 잠금이 없다면 하나의 커넥션에서 동시에 변경할 수 있게 되어 해당 레코드의 값은 예측이 불가능할 것이다
- 여러 커넥션에서 동시에 유일한 자원을 요청할 경우 순서대로 한 시점에는 하나의 커넥션만 변경할 수 있게 해주는 역할을 한다

트랜잭션 (Transaction)

- 작업의 완전성(원자성)을 보장
- 여러 작업을 하나의 논리적 단위로 묶어서 전체 성공 또는 전체 실패를 보장

격리 수준 (Isolation Level)

- 하나의 트랜잭션 내에서 또는 여러 트랜잭션 간의 작업 내용을 어떻게 공유하고 차단할 것인지 결정하는 레벨이다
-

5.1 트랜잭션

여러 SQL 명령을 하나로 묶어 전체 성공 또는 전체 실패하게 보장하는 기능이다.

ACID 특징

트랜잭션은 다음과 같은 4가지 특징을 가지고 있다.

- **Atomicity(원자성)**: 전부 성공 혹은 전부 실패
- **Consistency(일관성)**: 데이터 무결성 항상 유지
- **Isolation(고립성)**: 트랜잭션끼리 서로 간섭하지 않게 격리
- **Durability(지속성)**: commit되면 디스크에 확실히 기록한다

트랜잭션의 내부 동작 (InnoDB)

InnoDB 스토리지 엔진에서 트랜잭션이 어떻게 동작하는지 단계별로 살펴보자.

- 1) **Buffer Pool**에 페이지 로딩
 - SELECT, UPDATE 발생 시 데이터 페이지가 디스크 → 버퍼 풀로 로드된다
- 2) 변경 시 **Undo Log** 생성 (MVCC 기반)
 - UPDATE/DELETE/INSERT 등 쓰기 작업 발생할 때 InnoDB는 먼저 **Undo Log**를 생성한다
 - rollback 시 이전 상태를 복원한다
 - MVCC의 **consistent read**를 위해 제공한다
- 3) 데이터 변경 → 버퍼 풀에 **Dirty Page** 생성
 - 데이터는 즉시 디스크에 쓰지 않는다
 - 메모리(Buffer Pool)에서 **Dirty Page**로 표시된다
- 4) **Redo Log**에 변경 내용 기록
 - 변경 내용은 Redo Log 버퍼 → Redo Log 파일 순서로 기록된다
- 5) Commit → Redo Log에 commit 마크 기록
 - 커밋하면 Redo Log에 **commit record**가 찍힌다
- 6) 디스크 반영은 나중에
 - InnoDB는 체크포인트 알고리즘을 따라 **Dirty Page**를 백그라운드 스레드가 나중에 디스크에 쓴다

이러한 메커니즘을 통해 InnoDB는 성능과 안정성을 동시에 확보한다. 메모리에서 먼저 작업하고, Redo Log로 복구 가능성을 보장하며, 실제 디스크 쓰기는 효율적으로 배치 처리한다.

5.1.1 MySQL의 트랜잭션

트랜잭션은 논리적인 작업 셋 자체가 100% 적용되거나(COMMIT 실행했을 때) 아무것도 적용되지 않아야 (ROLLBACK 또는 트랜잭션을 Rollback 시키는 오류 발생했을 때) 함을 보장해주는 것이다.

InnoDB 테이블과 MyISAM 테이블 차이

두 스토리지 엔진이 트랜잭션을 어떻게 다르게 처리하는지 실험을 통해 알아보자.

실험 설정

1. tab_myisam, tab_innodb 테이블을 만든다
2. 각 테이블에 fdpk(Primary Key) 컬럼만 생성하고 각각 3을 INSERT 한다
3. 각 테이블의 fdpk 컬럼에 대하여 1, 2, 3을 INSERT 하고 결과를 비교한다
4. 1, 2, 3을 넣을 때 이미 3이 있다고 둘 다 오류가 뜨지만 각 테이블의 결과를 확인한다

실험 결과

- tab_myisam : 1, 2, 3이 전부 있다
- tab_innodb : 3만 있다

결과 분석

두 INSERT 문장 모두 PK 중복 오류로 쿼리가 실패했다.

하지만 MyISAM 테이블에는 1, 2가 INSERT된 상태로 남아있다. 차례대로 1, 2를 저장한 후 3을 저장하는 순간 오류가 발생했으며, 이미 INSERT된 1, 2는 그대로 두고 쿼리 실행을 종료한 것이다.

반면 InnoDB는 일부라도 오류가 발생하면 전체를 원 상태로 만든다는 트랜잭션 원칙대로 실행되기 전 상태 그대로 복구되었다.

부분 업데이트 (Partial Update)

MyISAM 테이블에서 발생하는 이러한 현상을 **부분 업데이트**라고 표현한다.
이는 데이터의 정합성을 맞추는데 어려움을 겪게 한다.

5.1.2 주의 사항

트랜잭션 또한 DBMS의 커넥션과 동일하게 꼭 필요한 최소한의 코드에만 적용을 해야 한다.

트랜잭션 범위 최소화하기

다음은 게시물 등록 프로세스를 예시로 트랜잭션 범위를 어떻게 설정해야 하는지 알아보자.

잘못된 예시

- 1) 처리 시작
 - DB 커넥션 생성
 - 트랜잭션 시작
- 2) 사용자 로그인 여부 확인
- 3) 사용자 글 쓰기 내용 오류 여부 확인
- 4) 첨부로 업로드된 파일 확인 및 저장
- 5) 사용자 입력 내용 DBMS에 저장
- 6) 첨부파일 정보 DBMS에 저장
- 7) 저장된 내용 또는 기타 정보 DBMS에서 조회
- 8) 게시물 등록에 대한 알림 메일 발송
- 9) 알림 메일 발송 이력 DBMS에 저장
 - ← 트랜잭션 종료 (COMMIT)

위 예제에는 문제점들이 있다. 한번 살펴보도록 하자.

문제점 1: 불필요하게 넓은 트랜잭션 범위

실제 DBMS에 데이터 저장 작업은 5번부터이며, 2, 3, 4번은 굳이 트랜잭션에 포함시킬 이유가 없다.

일반적으로 DB 커넥션은 개수가 제한적이라서 각 단위 프로그램이 커넥션을 소유하는 시간이 길어질수록 사용 가능한 커넥션 개수가 줄어든다. 이는 전체 시스템의 성능 저하로 이어진다.

문제점 2: 네트워크 통신 작업 포함

메일 전송, FTP 파일 전송, 네트워크를 통해 원격 서버와 통신하는 작업은 **어떻게 해서든 DBMS의 트랜잭션 내에서 제거해야 한다.**

만약 통신 불가한 상황이 발생하면 웹 서버뿐만 아니라 DB 서버까지 위험해진다. 트랜잭션이 열린 상태로 오랜 시간 대기하게 되면 락이 유지되고, 다른 요청들이 블로킹되어 전체 시스템이 마비될 수 있다.

개선된 예시

5, 6번 작업은 반드시 하나의 트랜잭션으로 묶고, 9번은 별도의 트랜잭션으로 분리한다.

- 1) 처리 시작
- 2) 사용자 로그인 여부 확인
- 3) 사용자 글 쓰기 내용 오류 여부 확인
- 4) 첨부로 업로드된 파일 확인 및 저장
 - DB 커넥션 생성
 - 트랜잭션 시작
- 5) 사용자 입력 내용 DBMS에 저장
- 6) 첨부파일 정보 DBMS에 저장
 - ← 트랜잭션 종료 (COMMIT)
- 7) 저장된 내용 또는 기타 정보 DBMS에서 조회
- 8) 게시물 등록에 대한 알림 메일 발송
 - 트랜잭션 시작
- 9) 알림 메일 발송 이력 DBMS에 저장
 - ← 트랜잭션 종료 (COMMIT)
 - ← DB 커넥션 반납
- 10) 처리 완료

트랜잭션 사용 시 구체적인 주의사항

이제 트랜잭션의 주의사항에 대해서 좀 더 구체적으로 알아보자.

1) 트랜잭션은 짧게 유지해야 한다

트랜잭션을 오래 잡고 있으면 락을 오래 잡고 있는 것과 동일하다

문제점

- Row Lock / Next-Key Lock이 오랫동안 유지된다
- 다른 트랜잭션이 기다리다가 Lock Wait Timeout 발생
- 데드락 증가
- 커넥션 풀 고갈
- 전체 DB 지연 → 서비스 장애

해결 방법

DB 관련 작업만 트랜잭션 내부에 두고 외부 작업은 밖으로 빼자.

```
// 나쁜 예
@Transactional
public void processOrder() {
    validateUser();           // 트랜잭션 불필요
    checkInventory();         // 트랜잭션 불필요
    saveOrder();              // 트랜잭션 필요
    sendEmail();              // 트랜잭션 불필요
}

// 좋은 예
public void processOrder() {
    validateUser();
    checkInventory();

    saveOrderWithTransaction(); // 트랜잭션 필요한 부분만

    sendEmail();
}
```

2) 트랜잭션 안에서 외부 I/O 작업 금지

Cloudinary, Google/Naver API 요청 등 외부 I/O 작업이 트랜잭션 안에 들어가면 문제가 발생한다

문제점

- DB 트랜잭션이 불필요하게 유지된다
- DB 락을 장기간 점유한다
- 동시성이 떨어지고 전체 성능이 떨어진다
- 외부 서비스 장애 시 DB까지 영향을 받는다

해결 방법

DB 반영 → COMMIT → 외부 처리 순서로 분리하자.

```
// 나쁜 예
@Transactional
```

```

public void uploadImage(Image image) {
    saveImage(image);
    String url = cloudinaryService.upload(image); // 외부 API 호출
    updateImageUrl(url);
}

// 좋은 예
public void uploadImage(Image image) {
    saveImageWithTransaction(image); // 트랜잭션 처리

    String url = cloudinaryService.upload(image); // 외부 API 호출

    updateImageUrlWithTransaction(url); // 별도 트랜잭션
}

```

3) @Transactional은 기본적으로 Checked Exception에서 Rollback을 하지 않는다

Spring의 기본 규칙을 이해해야 한다.

Spring 트랜잭션 롤백 규칙

- **RuntimeException, Error**: Rollback 가능
- **Checked Exception**: Rollback 불가능

예시

```

// Rollback 안됨
@Transactional
public void process() throws IOException {
    saveData();
    if (error) throw new IOException(); // ROLLBACK 불가
}

// Rollback 됨
@Transactional(rollbackFor = Exception.class)
public void process() throws IOException {
    saveData();
    if (error) throw new IOException(); // ROLLBACK 가능
}

```

해결 방법

명시적으로 롤백 대상 예외를 지정하거나, RuntimeException으로 감싸서 던지자.

4) 하나의 트랜잭션에 너무 많은 쿼리를 넣지 말자

대량의 데이터를 한 트랜잭션에서 처리하면 여러 문제가 발생한다.

문제점

- Undo Log 폭증 → Purge Thread 밀림
- Redo Log 폭증 → Checkpoint Stall
- Dirty Page 증가 → Buffer Flush 지연 → 전체 서버 지연
- 테이블/행 락 장기 보유

해결 방법

- Batch Insert (500-1000건씩)
- 트랜잭션 분할
- 대량 작업은 비동기 워커/배치 서버로 분리

```
// 나쁜 예
@Transactional
public void insertMillion() {
    for (int i = 0; i < 1_000_000; i++) {
        repository.save(new Data(i));
    }
}

// 좋은 예
public void insertMillion() {
    List<Data> batch = new ArrayList<>();
    for (int i = 0; i < 1_000_000; i++) {
        batch.add(new Data(i));

        if (batch.size() == 1000) {
            saveBatchWithTransaction(batch);
            batch.clear();
        }
    }
}
```

5) Service 계층에서만 트랜잭션 사용

트랜잭션의 위치는 매우 중요하다.

문제점

- **Controller에 두면**: 모든 HTTP 요청보다 이상한 범위로 묶인다
- **Repository에 두면**: 호출할 때마다 자동 COMMIT → Atomicity 깨진다
- **여러 Service 메소드 호출하는 경우**: 관리가 어렵다

올바른 방법

Service 계층 = 비즈니스 로직 + 트랜잭션 경계

```

// Controller - 트랜잭션 없음
@RestController
public class OrderController {
    public ResponseEntity createOrder() {
        orderService.createOrder(); // Service에서 트랜잭션 관리
        return ResponseEntity.ok();
    }
}

// Service - 트랜잭션 있음
@Service
public class OrderService {
    @Transactional
    public void createOrder() {
        orderRepository.save(order);
        inventoryRepository.update(product);
    }
}

// Repository - 트랜잭션 없음
@Repository
public class OrderRepository {
    public void save(Order order) {
        // 단순 저장 로직만
    }
}

```

6) 트랜잭션 내 순서가 다른 UPDATE 주의

트랜잭션 내 순서가 다른 UPDATE는 데드락을 유발한다

문제 상황

둘 다 UPDATE는 하지만, 검색 조건 범위가 다르면 데드락이 발생한다.

```

-- 트랜잭션 1
UPDATE user SET status = 'active' WHERE id > 10;

-- 트랜잭션 2
UPDATE user SET status = 'inactive' WHERE id > 5;

-- 검색 범위가 겹쳐서 Next-Key Lock 충돌 발생

```

동작 원리

1. T1이 id > 10 범위에 락을 건다
2. T2가 id > 5 범위에 락을 걸려고 시도한다
3. 5~10 구간에서 충돌이 발생한다

4. 서로 상대방이 잡고 있는 락을 기다리면서 데드락 발생

해결 방법

- UPDATE 순서를 통일한다
- 특정 컬럼 기준 오름차순 정렬 후 UPDATE
- WHERE 조건을 제대로 설정해서 빠르게 인덱스를 타도록 튜닝한다

```
// 좋은 예 - 정렬 후 UPDATE
@Transactional
public void updateUsers(List<Integer> ids) {
    Collections.sort(ids); // 정렬 해서 순서 통일

    for (Integer id : ids) {
        userRepository.updateById(id);
    }
}
```

정리

트랜잭션은 데이터의 일관성과 무결성을 보장하는 중요한 기능이지만, 잘못 사용하면 오히려 성능 저하와 장애의 원인이 된다.

핵심 원칙

- 트랜잭션은 짧게 유지한다
- DB 작업만 트랜잭션 안에 넣는다
- 외부 I/O는 트랜잭션 밖으로 뺀다
- Service 계층에서 트랜잭션을 관리한다
- 대량 작업은 분할하거나 별도 처리한다

이러한 원칙들을 지키면 안정적이고 효율적인 트랜잭션 관리가 가능하다.